

CSE233 Week 9: C Strings and STL Strings

There are two types of strings that you will encounter in C++, C strings and STL Strings. So far, we have used the STL String (the `std::string` type) for storing text data. But, C++ is built on the C language, and inherits its representation of strings as well. `std::string` itself is a wrapper around the C string type.

C Strings

The `char` data type in C and C++ is internally an integer. The character with an integer value of 0 is called the null terminator. It is also represented as `'\0'`. This is important for understanding C strings. All C strings end with the null terminator. A C string is an array of `char`s that ends with a null terminator. Below is an example C string:

```
char cStr[] = "Hello World";
```

Note that the null terminator does not need to be explicitly added. The compiler supports this shorthand syntax for creating a C string directly. If we created the array normally, such as with an initializer list, then we would have to manually include the null terminator.

```
// manually making the last character 0
char cStr[] = {'H', 'e', 'l', 'l', 'o', 0 };
```

C strings cannot be assigned to after they are created. For example, the code below is illegal:

```
char cStr[] = "Hello";
// illegal, cannot reassign a C string
cStr = "Hello World";
```

C strings also do not support any operators. They cannot be

concatenated (added) using the `+` operator, nor compared using `==`. C strings can be read from the console. To declare an empty C string, you just declare a character array. However, keep in mind that the size of the array needs to be one more than the number of characters the string has to store. This is due to the last character of the array needing to be dedicated to the null terminator.

```
// this C string can hold 15 characters
char cStr[16];
std::cout << "Enter some text (15 or less characters): ";
std::cin >> cStr;
std::cout << "You entered: " << cStr << "\n";
```

If the user enters a C string longer than 15 characters, the input will be truncated to 15 characters. For example:

```
Enter some text (15 or less characters): thisismorethan15characters
You entered: thisismorethan1
```

C String Member Functions

There are functions available in the `cstring` library for operating on strings. To use them, `#include <cstring>` in your file. Below are the functions provided by `cstring`. All the functions described here are provided already by `cstring`, but example definitions will also be provided to demonstrate how to operate on C strings. To be honest, I don't really recommend using the C string library. It is inconsistently implemented across compilers and operating systems, and even the best implementation is not very good. But, since these can be encountered in the wild, we will look at them.

`strlen()`

The `strlen()` function returns the length of a C string. The length does not include the null terminator. For example:

```
int length = strlen("Hello World");
assert(length == 11);
```

Below is a possible implementation of `strlen`:

```

unsigned long strlen(const char str[]) {
    unsigned long i = 0;
    // checking each character until a null terminator is found
    while (str[i] != 0) ++i;
    return i;
}

```

It is possible that a C string is malformed (i.e., it is missing the null terminator). More recent versions of C/C++ offer `strnlen`, with an `n` between `str` and `len`. This allows a max length to be passed to `strlen`.

```

// manually creating a malformed C string
char cStr[] = "Hello World";
// overwriting null terminator
cStr[11] = '\n';
// this will search until it finds a byte with
// the value 0, which will not be the correct value
// theoretically, it could infinite loop
int length = strlen(cStr);
// this may not be the correct length, but will never
// infinite loop. it will stop once it exhausts 1000 characters
int safeLength = strnlen(cStr, 1000);

```

Here is a possible implementation of `strnlen()`

```

unsigned long strnlen(const char str[], unsigned long n) {
    unsigned long i;
    while (str[i] != 0 && i <= n) ++i;
    return i;
}

```

strcpy()

The function `strcpy()` takes two C strings as arguments. The second argument is copied into the first. If the first argument has data already, it is overwritten. Here is an example of its usage:

```

char cStr[] = "Hello World";
char copy[12];
strcpy(cStr, copy);
std::cout << copy << "\n";
// prints: Hello World

```

Here is a potential implementation of `strcpy()`:

```
void strcpy(char destination[], const char source[]) {
    unsigned long length = strlen(source);
    // using <= to copy the null terminator too
    for (unsigned long i = 0; i <= length; ++i) {
        destination[i] = source[i];
    }
}
```

Like `strlen`, there is also a "safe" version of `strcpy()`, called `strncpy()`. This version takes a third argument, `n`, for the maximum number of characters to copy over. This way, if the source string is larger than the destination buffer, `strncpy()` will not write data out of bounds.

```
char longString[] = "This string is much longer than the destination";
char shortString[10];
// need to leave room for null terminator
strncpy(shortString, longString, 9);
// shortString is now "This stri", but does not have a null terminator
// manually insert null terminator
shortString[9] = 0;
```

`strncpy` does not add a null terminator if it runs out of characters to copy. At this point, the output string is simply a character array, and should not be treated as a C string anymore. Here is a safer way to check use `strncpy()`. Here, we check if the string being copied is longer than the destination. If it is, we copy as many characters as we can while still leaving a space for the null terminator. Otherwise, we copy the string normally.

```
char source1[] = "This string is shorter than its destination";
char source2[] = "This string is longer than its destination";

const int dest1Len = 44;
const int dest2Len = 16;

// we can use these variables as the sizes
// of the arrays since they are const
char dest1[dest1Len];
char dest2[dest2Len];

// check if the length of the source is less than
```

```

// the max length of the destination
if (strlen(source1, dest1Len) < dest1Len) {
    // if it is, we can copy normally
    strcpy(dest1, source1);
}
else {
    // otherwise, we need to manually add the null terminator
    strncpy(dest1, source1, dest1Len - 2);
    dest1[dest1Len - 1] = 0;
}

if (dest2Len < strlen(source2, dest2Len)) {
    strcpy(dest2, source2);
}
else {
    strncpy(dest2, source2, dest2Len - 2);
    dest2[dest2Len - 1] = 0;
}

```

Here is a better implementation of `strncpy()` that inserts a null terminator if the source string is longer than the destination:

```

void strncpy(char destination[], const char source[], unsigned long n)
{
    unsigned long sourceLength = strlen(source, n);
    // if the source string is shorter than n, we can copy it normally
    if (sourceLength < n) {
        for (unsigned long i = 0; i <= sourceLength; ++i) {
            destination[i] = source[i];
        }
    }
    // otherwise, copy as many characters as we can, then insert null
    else {
        for (unsigned long i = 0; i < n - 1; ++i) {
            destination[i] = source[i];
        }
        destination[n - 1] = 0;
    }
}

```

strcat()

The `strcat()` function is used to concatenate (i.e., add) two strings. The first string passed will be modified. The result is the first string with the second string appended to it. The first string must be a large enough array to hold itself and the second string. The second string will not be modified by the function.

```

char str1[] = "Hello ";
char str2[] = "World";
// "Hello World" is 11 characters, + 1 for null terminator
char result[12];
// first, we need to copy str1 into result
strcpy(result, str1);
// then concatenate str2 onto the end of result
strcat(result, str2);

```

Here is a potential implementation of `strcat()`:

```

void strcat(char destination[], const char source[]) {
    unsigned long destLen = strlen(destination);
    unsigned long srcLen = strlen(source);

    // start copying at the end of the destination,
    // overwriting it's null terminator with the first char of source
    // again, using <= to copy source's null terminator
    for (unsigned long i = destLen + 1; i <= srcLen; ++i) {
        destination[i] = source[i - srcLen];
    }
}

```

Like the other C string functions, there is also an `n` version of `strcat()`. The `strncat()` function has an additional argument for the maximum number of characters to copy from the source string. Like `strncpy()`, `strncat()` does not insert a null terminator if the source string is larger than `n`. Therefore, like `strncpy()`, we need to be careful with how we use this function, as it may not "return" a valid C string. We can use the same guards as before to ensure that the result becomes a valid C string:

```

const int resultSize = 30;
char str1[] = "Start of string.";
char str2[] = "Rest of string.";
// this is not quite large enough to hold both str1 and str2
char result[resultSize];
strcpy(result, str1);
// need to check if we can copy all of str2
unsigned long str1Len = strlen(str1);
unsigned long str2Len = strlen(str2);
if (str2Len < (resultSize - str1Len - 1)) {
    strncat(result, str2, (resultSize - str1Len - 1));
    result[resultSize - 1] = 0;
}

```

```
else {  
    strcat(result, str2);  
}
```

Here is a better implementation of `strncat()` that does insert a null terminator:

```
void strncat(char destination[], const char source[], unsigned long n)  
{  
    unsigned long destLength = strlen(destination, n);  
    unsigned long srcLength = strlen(source, n);  
  
    // if the destination length + null character is less than n  
    // we can start concatenating the string  
    if (destLength + 1 < n) {  
        // compute the number of characters we can copy  
        // need to subtract an extra 1 for the null terminator  
        unsigned long maxChars = n - destLength - 1;  
        if (srcLength < maxChars) {  
            maxChars = srcLength;  
            for (unsigned long i = destLength; i <= maxChars; ++i) {  
                destination[i] = source[i - srcLength];  
            }  
        }  
        else {  
            for (unsigned long i = 0; i < maxChars; ++i) {  
                destination[i + destLength] = source[i];  
            }  
            destination[n - 1] = 0;  
        }  
    }  
    // otherwise, we can't do anything since the destination is already full  
}
```

strcmp()

`strcmp()` is used to compare two strings. It returns an integer. The strings are compared lexicographically (dictionary order). However, in C++ (and most programming languages), all capital letters come before lowercase. So, capital 'Z' comes before lowercase 'a'. `strcmp()` returns a negative number if the first string comes before the second, 0 if the strings are the same, and a positive number if the first string comes after the second. Below are some examples:

```
char str1[] = "abc";  
char str2[] = "ABC";
```

```

char str3[] = "xyz";
char str4[] = "Hello";
char str5[] = "hello";
char str6[] = "hello";
char str7[] = "abcd";

int result;

// result is 0
result = strcmp(str1, str1);
// result is 0
result = strcmp(str5, str6);
// result is positive
result = strcmp(str1, str2);
// result is negative
result = strcmp(str2, str1);
// result is positive
result = strcmp(str3, str2);
// result is negative
result = strcmp(str2, str3);
// result is negative
result = strcmp(str4, str5);
// result is positive
result = strcmp(str5, str4);
// result is negative
result = strcmp(str1, str7);

```

Below is a possible implementation of `strcmp()`:

```

int strcmp(const char str1[], const char str2[]) {
    unsigned long i = 0;

    while (str1[i] != 0 && str2[i] != 0) {
        if (str1[i] < str2[i]) return -1;
        else if (str1[i] > str2[i]) return 1;
        ++i;
    }

    if (str1[i] == 0) return -1;
    if (str2[i] == 0) return 1;
    // if we got here all the characters were the same
    // and the strings are the same length
    return 0;
}

```

There is also the `n` version of `strcmp()`, `strncmp()`. This version will compare up to `n` characters of the strings. It stops comparing if a null terminator is found in either string:


```
char str1[] = "hello"
char str2[] = "hello world";

int result;
// result is negative
result = strncmp(str1, str2, 11);
// result is 0
result = strncmp(str1, str2, 5);
```

Below is a possible implementation of `strncmp()`:

```
int strncmp(const char str1[], const char str2[], unsigned long n) {
    if (n == 0) return 0;

    unsigned long i = 0;

    while (str1[i] != 0 && str2[i] != 0 && i <= n) {
        if (str1[i] < str2[i]) return -1;
        else if (str1[i] > str2[i]) return 1;
        ++i;
    }

    if (str1[i] == 0) return -1;
    if (str2[i] == 0) return 1;
    // if we got here all the characters were the same
    // and the strings are the same length
    return 0;
}
```

STL Strings

The `std::string` type is defined as part of C++'s Standard Template Library (STL). C++ strings are still C strings internally, but make working with strings significantly easier. STL strings support allow many operations which make strings easier to use. In the following subsections, we'll discuss the various string operations.

Objects

`std::string` is a class. You should have had a brief introduction to classes at the end of programming logic. In short, a class is data type that groups data and operations (member functions) into a single type. An object is an instance (i.e., variable) of a class. Each string variable you create is a string object. Class member functions are called for an object using the dot operator in C++.

Assigning and Copying

A string can be declared like any other variable, except the `string` header needs to be included.

```
std::string str;
```

A string can also be created using the string constructor. Both string literals and string objects can be passed to the constructor. The constructor is only called when the variable is created.

```
// calling constructor with a string literal
str::string str("Hello world");
// call constructor with a string variable
std::string copy(str);
```

Strings can also be created and copied using the `=` operator:

```
std::string str = "Hello World";
std::string copy = str;
```

Concatenation

Strings can be concatenated using the `+` and `+=` operators, as well as the `append()` member function. The `+=` operator and `append()` function modify an existing string, while the `+` operator returns a new string that is a combination of the two strings being concatenated:

```
std::string str = "Hello";
str += " world";
// str is now "Hello world"

// append is a member function
// use the . operator to call it
str.append("!\n")
// str is now "Hello world!\n"

std::string str2 = "Hello" + " world";
// str2 is "Hello world"
```

Comparison

Strings can be compared using the normal comparison operators like `==`, `<`, `>`, etc. Like C strings, `<` and `>` compare the strings lexicographically. Just like C strings, the comparison is really based on the order that the characters appear in ASCII order, so uppercase letters are "smaller" than lowercase. Certain characters like punctuation are smaller than letters.

Length, Empty, Subscript, and `at()`

The `length()` member function is used to access the length of the string. It returns the number of characters in the string. The `empty()` member function returns `true` if the string is empty, and `false()` otherwise. Like a C string, the individual characters of the string can be accessed using the subscript operator. A string can be treated just like an array of characters. The `at()` member function is used in the same way as the subscript operator, except it will throw an exception if an out-of-range access is attempted.

```
std::string example = "This is an example std::string";
for (int i = 0; i < example.length(); ++i) {
    // print each character on it's own line
    std::cout << example[i] << std::endl;
}
std::string emptyString = "";
if (emptyString.empty()) {
    // this will print, emptyString holds no characters
    std::cout << "emptyString is empty\n";
}
// use at() to access an individual characer
example.at(0) = 't';
// this will crash the program immediately,
// since the index passed is out of range
example.at(100) = '!';
```

Find, Find First Of, and Substring

The `find()` member function is used to find the position of a string within another string. It returns the index of the first occurrence of the string being searched for. The member function takes two arguments. The first is the string to search for. The second is the position to start searching from. This parameter has a default value of 0 (starts searching from the start of the string). A variation of `find()` called `rfind()` exists to search from the back of the string instead of the front.

`rfind()` will return the start index of the last instance of the string being searched for. Both versions return a special value, `std::string::npos`, when the target string is not found. `std::string::npos` is the

```
std::string example = "This string is an example";

// pos == 5
int pos = example.find("string");
// pos == std::string::npos
pos = example.find("string", 6)
// pos == 2
pos = example.find("is");
// pos == 12
pos = example.rfind("is");
// pos == 2
pos = example.rfind("is", 14);
```

The `find_first_of()` function passed a string of characters to search for. The index of the first occurrence of one of the characters is returned. If none of the characters are found in the string, `std::string::npos` is returned. The `find_last_of()` does the same thing as `find_first_of()`, except it starts searching from the end of the string. Like `find()`, a position to start searching from can be passed as the second argument.

```
std::string example = "This is an example string. It contains punctuation";
std::string punctuation = ",.?!;:";
// pos == 25, finds the first period
int pos = example.find_first_of(punctuation);
// pos == 50, finds the last period
pos = example.find_last_of(punctuation);
```

There are also equivalent `find_first_not_of()` and `find_last_not_of()` functions, which return the first (or last) index of the first character that does not match the argument.

Erase, Substring, and Replace

The `erase()` member function takes two arguments, a start index and the number of characters to remove, starting at the start index.

`erase()` modifies the string that calls it. The `substr()` member function returns a substring of the string it is called on. It, like `erase()`,

expects the start index and number of characters to extract. Finally, the `replace()` member function expects three arguments: the start index, the number of characters to replace, and the string to replace them with.

```
std::string example = "This is an example std::string";
// example will be "This is an example string"
example.erase(19, 5);
// example will be "This is an example STL String"
example.replace(19, 1, "STL S");
// substring will be "example"
std::string example = example.substr(11, 7);
```

Example

Pig Latin has the following rules:

- Words that start with a vowel have "way" appended to the end
- Words that start with a consonant have the consonant moved to the end, and "ay" appended to the end

Let's write a function that converts a word to Pig Latin:

```
const std::string VOWELS = "aeiouAEIOU";
const std::string EXTENDED_VOWELS = "aeiouyAEIOUY";
const std::string UPPERCASE = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

// helper function to check if character is upper case
bool isUpper(char c) {
    if (UPPERCASE.find(c) != std::string::npos) {
        return true;
    }
    else {
        return false;
    }
}

// helper function to check if string is uppercase
bool isUpper(const std::string str) {
    for (int i = 0; i < str.length(); ++i) {
        if (!isUpper(str[i])) return false;
    }
    return true;
}

void convertToPigLatin(std::string& word) {
    // 'y' can sometimes be a vowel
```

```

// it's not a perfect check, but we'll
// assume that if the first letter is not
// a vowel, then y is allowed to be a vowel

// first, check if the first letter is a vowel
if (VOWELS.find(word[0]) != std::string::npos) {
    // the first letter is a vowel, append way
    if (isUpper(word)) {
        word.append("WAY");
    }
    else {
        word.append("way");
    }
}
else {
    // the first letter is not a vowel
    // first, extract and remove any leading consonants
    int end = word.find_first_of(EXTENDED_VOWELS);
    // if this is true, the "word" had no vowels
    if (end == std::string::npos) {
        if (isUpper(word)) {
            word.append("AY");
        }
        else {
            word.append("ay");
        }
    }
    else {
        // copy the consonants
        // there could be more than one, like "th", "sh", etc.
        std::string start = word.substr(0, end);
        word.erase(0, end);
        // check if the word was all caps
        if (isUpper(word) && isUpper(start)) {
            word.append(start);
            word.append("AY");
        }
        // check if the word started with a capital letter
        else if (isUpper(start[0])) {
            // make consonant lowercase
            start[0] += 32;
            // make start of word uppercase
            word[0] -= 32;
            word.append(start);
            word.append("ay");
        }
        else {
            word.append(start);
            word.append("ay");
        }
    }
}
}

```

```
}  
}
```

We can also define a function that converts a sentence to Pig Latin, using the function that converts single words to Pig Latin. For this example, we'll assume that the sentence contains no punctuation.

```
void convertSentenceToPigLatin(std::string& sentence) {  
    int start = 0;  
    // find the first space  
    int end = sentence.find(" ");  
    std::string word;  
    // while a space is still found  
    while (end != std::string::npos) {  
        // extract the word  
        word = sentence.substr(start, end - start);  
        // convert the extracted word to pig latin  
        convertToPigLatin(word);  
        // replace the original word with the pig latin word  
        sentence.replace(start, end - start, word);  
        // update the start position to be  
        // one more than the end of the new word  
        start = end + 1 + (word.length() - (end - start));  
        // find the next space  
        end = sentence.find(" ", start);  
    }  
    // handle the last word  
    word = sentence.substr(start, std::string::npos);  
    convertToPigLatin(word);  
    sentence.replace(start, sentence.length() - start, word);  
}
```

Conclusion

This week we discussed strings, both the original C strings and C++ STL strings. C strings still exist for backwards compatibility with C, but the C string library is lackluster. Most of the time, using a C++ STL string is advisable. The C++ string is implemented as a class. Variables of classes are called objects, and objects have functions associated with them. C++ strings have many member functions associated with its objects, as well as support for operators.